Textbook Generation Agent — Product Proposal & Architecture Specification

Version: 1.0
Status: Ready for Implementation
Primary Language: Python
Target: AI Coding Agent Implementation Guide

---

1. Overview

The Textbook Generation Agent is a standalone, AI-agnostic pipeline that accepts a learner profile as input and produces a complete, rendered, high-quality textbook as output. It operates independently — no chatbot, no UI, no external integrations required. Once the engine is stable, it becomes a pluggable service that any interface can call.

The core thesis: most AI tutoring tools deliver the same content to everyone. This system generates a document that could not have existed before a specific learner profile was provided. The textbook is shaped by who is learning, what they already know, and how deeply they need to go.

1.1 What It Is Not (Phase 1 Scope Boundary)

- Not a chatbot or conversational interface
- Not integrated with any diagnostic system yet
- Not producing audio or runnable code cells yet
- Not a multi-user platform yet

These are Phase 2+ concerns. The Phase 1 deliverable is: **profile in → textbook out, reliably and at high quality.**

---

2. Input and Output Contract

2.1 Input — Learner Profile

The simplest viable profile. Intentionally minimal for Phase 1.

```
json
```

```
{
  "subject": "calculus",
  "age": 17,
  "context": "I understand derivatives and basic limits but integration confuses me. I have seen the chain rule but never really u
  "depth": "standard",
  "language": "math_notation"
}
```

Field definitions:

Field	Type	Values	Description
subject	string	any	The subject domain e.g. "calculus", "DSA", "linear algebra"
age	integer	8–99	Drives vocabulary complexity, example choices, tone
context	string	free text	What the learner knows, what they struggle with, any signals
depth	string	"survey" / "standard" / "deep"	Controls section depth and number of worked examples
language	string	"plain" / "math_notation" / "python" / "pseudocode"	Preferred notation style

2.2 Output — Rendered Textbook

- Primary format: **HTML** (self-contained, single file, no external dependencies)
- Secondary format: **PDF** (Phase 2)
- Metadata file: **JSON** describing structure, section count, generation time, model used

---

3. Architecture

3.1 Design Principles

Separation of concerns — three clear layers:

```
Application Layer   →  What the agent does (pipeline orchestration, CLI)
Service Layer       →  How it does it (nodes, prompts, rendering)
Infrastructure Layer →  What it runs on (LLM providers, file I/O, config)
```

No layer reaches past its neighbour. The application layer never calls an LLM directly. The infrastructure layer never knows about pedagogy.

AI-agnostic from day one. Every LLM call goes through an abstract `BaseProvider`. Swapping Claude for GPT-4 is a single config change. Adding a new model is implementing one interface.

Schema-driven. Every node has a typed input schema and a typed output schema enforced by Pydantic. A node cannot pass malformed data to the next node. This is what makes the pipeline reliable.

Quality-checked before delivery. The pipeline does not return until the quality checker passes. Bad output triggers targeted node reruns, not full regeneration.

3.2 Directory Structure

```
textbook_agent/
│
├── app/                    # APPLICATION LAYER
│   ├── __init__.py
│   ├── agent.py              # TextbookAgent — top-level orchestrator
│   └── cli.py               # CLI entry point (argparse)
│
├── services/               # SERVICE LAYER
│   ├── __init__.py
│   ├── pipeline/
│   │   ├── __init__.py
│   │   ├── node_base.py        # Abstract PipelineNode — all nodes inherit this
│   │   ├── planner.py          # Node 1: Curriculum Planner
│   │   ├── content_generator.py   # Node 2: Content Generator
│   │   ├── diagram_generator.py   # Node 3: SVG Diagram Generator
│   │   ├── code_generator.py      # Node 4: Code Example Generator
│   │   ├── assembler.py        # Node 5: HTML Assembler
│   │   └── quality_checker.py    # Node 6: Quality Checker
│   │
│   ├── prompts/
│   │   ├── __init__.py
│   │   ├── base_prompt.py       # Shared pedagogical rules — injected into every node
│   │   ├── planner_prompts.py
│   │   ├── content_prompts.py
│   │   ├── diagram_prompts.py
│   │   └── quality_prompts.py
```

```
│   │
│   └── renderer/
│       ├── __init__.py
│       ├── html_renderer.py       # Assembles final HTML from section outputs
│       └── assets/
│           ├── base.css           # Design system — single source of truth
│           └── prism.css          # Code syntax highlighting
│
├── infra/                         # INFRASTRUCTURE LAYER
│   ├── __init__.py
│   ├── providers/
│   │   ├── __init__.py
│   │   ├── base.py                # Abstract BaseProvider
│   │   ├── factory.py             # ProviderFactory.get("claude") / ("openai")
│   │   ├── anthropic_provider.py  # Claude implementation
│   │   └── openai_provider.py     # GPT-4 implementation
│   │
│   └── storage/
│       ├── __init__.py
│       └── file_storage.py        # Reads profiles, writes outputs
│
├── schemas/                       # SHARED CONTRACTS (used by all layers)
│   ├── __init__.py
│   ├── learner_profile.py         # Input schema
│   ├── curriculum_plan.py         # Planner output schema
│   ├── section.py                 # Single section schema
│   └── textbook.py                # Final assembled textbook schema
│
├── config/
│   ├── __init__.py
│   └── settings.py                # Pydantic Settings — reads from .env
│
├── tests/
│   ├── __init__.py
│   ├── fixtures/
│   │   ├── stem_beginner.json
│   │   ├── stem_intermediate.json
│   │   └── stem_advanced.json
│   ├── test_schemas.py
│   ├── test_providers.py
│   ├── test_pipeline.py
│   └── test_renderer.py
│
├── outputs/                       # Generated textbooks land here (gitignored)
```

```
│
├── main.py                    # Entry point
├── requirements.txt
├── .env.example
└── README.md
```

---

4. The Pipeline

Each node is a `PipelineNode`. Each has:

- `input_schema` — Pydantic model, validated before run
- `output_schema` — Pydantic model, validated after run
- `run(input)` → output
- `retry_on_failure` — bool (default True, max 2 retries)

The agent chains nodes sequentially. If a node fails validation after retries, the pipeline raises a `PipelineError` with the node name and failure reason.

```
LearnerProfile
   ↓
[Node 1] Planner         →  CurriculumPlan
   ↓
[Node 2] ContentGenerator →  list[SectionContent]   (runs per section)
   ↓
[Node 3] DiagramGenerator →  list[SectionDiagram]   (runs per section needing visual)
   ↓
[Node 4] CodeGenerator    →  list[SectionCode]      (runs per section needing code)
   ↓
[Node 5] Assembler        →  RawTextbook            (pure Python, no LLM)
   ↓
[Node 6] QualityChecker   →  QualityReport          (LLM validates structure + prose)
   ↓
  PASS → HTMLRenderer → final output
  FAIL → re-run flagged nodes → check again → deliver or raise
```

4.1 Node Specifications

Node 1 — Curriculum Planner

Input: `LearnerProfile`
Output: `CurriculumPlan`

Responsibilities:

- Determine which topics to include based on subject, age, depth, and context
- Order topics respecting prerequisite dependencies (critical for STEM — cannot teach integration before derivatives)
- Flag each section as core vs supplementary
- Specify what each section needs: diagram, code example, or both
- Output a table of contents with metadata per section

CurriculumPlan schema:

```python
class SectionSpec(BaseModel):
    id: str                     # e.g. "section_03"
    title: str
    learning_objective: str
    prerequisite_ids: list[str]     # must appear before this section
    needs_diagram: bool
    needs_code: bool
    is_core: bool                   # False = supplementary, can be skipped at survey depth
    estimated_depth: Literal["light", "medium", "deep"]

class CurriculumPlan(BaseModel):
    subject: str
    total_sections: int
    sections: list[SectionSpec]
    reading_order: list[str]        # ordered list of section IDs
```

Node 2 — Content Generator

Input: `SectionSpec` + `LearnerProfile`
Output: `SectionContent`

Runs once per section. The pedagogical rules from `base_prompt.py` are injected into every call. These rules are invariants — they cannot be overridden by the section-level prompt.

Pedagogical invariants (baked into base_prompt.py):

- Never introduce a symbol before the learner feels the need for it
- Plain English before formal notation, always
- Every section opens with an intuition hook — a real situation that creates the problem
- Formal definition comes after the concept is felt, not before

- Every claim that could be misunderstood gets a concrete example
- Difficulty must not spike — each section assumes only what came before it

SectionContent schema:

```python
class SectionContent(BaseModel):
    section_id: str
    hook: str                    # opening paragraph — felt problem before named solution
    plain_explanation: str       # concept in plain language, no jargon
    formal_definition: str       # precise definition, notation where appropriate
    worked_example: str          # full example, every step explained
    common_misconception: str    # one thing learners consistently get wrong here
    connection_forward: str      # one sentence linking to next section
```

Node 3 — Diagram Generator

Input: `SectionSpec` + `SectionContent`
Output: `SectionDiagram`

Phase 1: SVG only. No external image API. Fully deterministic, no network dependency.
Phase 2: Add DALL-E / external API for sections needing richer visuals.

The diagram node receives the section content and decides what visual best serves the explanation. It generates SVG markup directly. The SVG must be self-contained (no external references).

SectionDiagram schema:

```python
class SectionDiagram(BaseModel):
    section_id: str
    svg_markup: str              # complete inline SVG
    caption: str                 # one sentence describing what the diagram shows
    diagram_type: str            # e.g. "number_line", "function_plot", "array_trace", "tree"
```

Node 4 — Code Generator

Input: `SectionSpec` + `SectionContent`
Output: `SectionCode`

Only runs for sections where `needs_code=True`. Generates a clean, runnable example with inline comments explaining what to observe. Code must be syntactically valid — the quality checker validates this.

SectionCode schema:

```python
class SectionCode(BaseModel):
    section_id: str
    language: str              # "python" for Phase 1
    code: str                  # syntactically valid, runnable
    explanation: str           # what to observe when running this
    expected_output: str       # what the code prints/returns
```

Node 5 — Assembler

Input: All section outputs (`SectionContent`, `SectionDiagram`, `SectionCode`)
Output: `RawTextbook`

Pure Python. No LLM call. Combines all outputs into a structured document object ready for rendering. Applies reading order from the curriculum plan.

Node 6 — Quality Checker

Input: `RawTextbook` + `CurriculumPlan`
Output: `QualityReport`

Validates:

- Every section in the plan is present in the textbook
- No section references a concept not introduced in a prior section
- Terminology is consistent (same term used for the same concept throughout)
- Difficulty progression is monotonic — no unexplained complexity spikes
- All code blocks are syntactically valid Python (via `ast.parse`)
- Worked examples match stated learning objectives

Returns `passed: bool` and `issues: list[Issue]`. If `passed=False`, the agent re-runs only the flagged nodes, not the full pipeline.

---

5. Provider Architecture

5.1 Abstract Base

```python
```

```python
# infra/providers/base.py
from abc import ABC, abstractmethod
from typing import Any

class BaseProvider(ABC):

    @abstractmethod
    def complete(
        self,
        system_prompt: str,
        user_prompt: str,
        response_schema: type,        # Pydantic model — provider returns validated instance
        temperature: float = 0.3,
        max_tokens: int = 4096,
    ) -> Any:
        """Make a completion call. Always returns a validated Pydantic instance."""
        ...

    @abstractmethod
    def name(self) -> str:
        """Return provider identifier string e.g. 'claude-3-5-sonnet'"""
        ...
```

5.2 Factory

```python
# infra/providers/factory.py
from .anthropic_provider import AnthropicProvider
from .openai_provider import OpenAIProvider

class ProviderFactory:
    _registry = {
        "claude": AnthropicProvider,
        "openai": OpenAIProvider,
    }

    @classmethod
    def get(cls, name: str) -> BaseProvider:
        if name not in cls._registry:
            raise ValueError(f"Unknown provider: {name}. Available: {list(cls._registry.keys())}")
        return cls._registry[name]()
```

5.3 Structured Output Requirement

Both providers must support **structured output** — returning a Pydantic model instance, not raw text. This is non-negotiable. Every node defines its output schema, and the provider is responsible for returning data that conforms to it.

- **Anthropic:** Use `response_format` with tool use / JSON mode
- **OpenAI:** Use `response_format={"type": "json_object"}` with schema in system prompt

The node never parses raw strings. If the provider cannot conform to the schema after retries, it raises a `ProviderConformanceError`.

---

6. Configuration

```python
```

```python
# config/settings.py
from pydantic_settings import BaseSettings
from typing import Literal

class Settings(BaseSettings):
    # Provider
    provider: Literal["claude", "openai"] = "claude"
    anthropic_api_key: str = ""
    openai_api_key: str = ""

    # Models
    claude_model: str = "claude-sonnet-4-6"
    openai_model: str = "gpt-4o"

    # Pipeline behaviour
    max_retries: int = 2
    quality_check_enabled: bool = True
    temperature: float = 0.3

    # Output
    output_dir: str = "outputs/"
    output_format: Literal["html", "pdf"] = "html"

    class Config:
        env_file = ".env"

settings = Settings()
```

---

7. Prompts Architecture

7.1 Base Prompt — Pedagogical Invariants

This is injected as a system-level preamble into every single node that generates content. It is the pedagogical DNA of the system. No node can override it.

```python
```

```
# services/prompts/base_prompt.py

BASE_PEDAGOGICAL_RULES = """
You are a world-class educator generating content for a personalized textbook.
These rules are invariants. They apply to every section you generate.

RULES:
1. Never introduce a symbol, term, or formula before the learner feels the need for it.
   The concept must arrive as the answer to a felt problem, not as a rule from authority.

2. Plain English always precedes formal notation.
   If you must use a symbol, the reader must already understand what it represents.

3. Every section opens with an intuition hook — a real, concrete situation
   that creates the exact problem this concept solves.

4. Difficulty must not spike. Each section may only assume knowledge
   explicitly covered in prior sections.

5. Every claim that could be misunderstood gets a concrete example immediately.

6. Tone matches the learner's age. Age 12 gets different vocabulary than age 22.
   Younger learners get more analogies. Older learners get more precision.

7. Never talk down to the learner. Simplicity and respect are not in conflict.
"""
```

7.2 Node Prompts

Each node has its own prompt file. Prompts are functions that take context and return a formatted string. They are not templates — they are code.

```python
```

```python
# services/prompts/planner_prompts.py

def build_planner_prompt(profile: LearnerProfile) -> str:
    return f"""
{BASE_PEDAGOGICAL_RULES}

You are planning the curriculum for a personalized {profile.subject} textbook.

LEARNER:
- Age: {profile.age}
- Depth requested: {profile.depth}
- Context: {profile.context}

Your task: produce a CurriculumPlan.

Requirements:
- Order topics so that no section requires knowledge not covered in a prior section.
- For STEM subjects, prerequisite ordering is critical — enforce it strictly.
- Mark sections as core or supplementary based on depth requested.
- Identify which sections need a diagram and which need a code example.
- For "survey" depth: include only core sections.
- For "standard" depth: include core + key supplementary sections.
- For "deep" depth: include everything.

Return your response as a valid JSON object conforming to the CurriculumPlan schema.
"""
```

---

8. Renderer

The HTML renderer is pure Python — no LLM, no external dependencies. It takes the assembled `RawTextbook` and produces a single self-contained HTML file.

8.1 Design System Principles

The visual design is a fixed asset (`base.css`). It does not change between textbooks. Consistency across all generated textbooks is a product requirement — teachers and students should recognise the format immediately.

Design language:

- Dark theme, high contrast, optimised for long reading sessions
- Monospace display font for headings and code
- Serif body font for prose — not sans-serif, reading speed matters

- Colour coding: green for definitions, orange for warnings/pitfalls, blue for examples, purple for connections
- Sticky sidebar navigation generated from section titles
- SVG diagrams inline — no external image requests

8.2 Section Template Structure

Every section renders identically:

1. Section header (day/unit number + title + subtitle)
2. Intuition hook (styled callout box)
3. Plain explanation (prose)
4. Formal definition (styled definition box)
5. Diagram (inline SVG, if present)
6. Worked example (step-by-step table)
7. Code example (syntax-highlighted, if present)
8. Common misconception (warning box)
9. Write-in practice area (input fields)

---

9. CLI Interface

```bash
# Basic usage
python main.py --profile tests/fixtures/stem_beginner.json

# With options
python main.py \
  --profile tests/fixtures/stem_intermediate.json \
  --provider claude \
  --depth deep \
  --output outputs/ \
  --format html

# Output
# outputs/calculus_age17_standard_20260311_143022.html
# outputs/calculus_age17_standard_20260311_143022_meta.json
```

---

10. Build Phases

Phase 1 — Core Engine (Build First)

Everything needed to go from profile to rendered textbook reliably.

Deliverables:

- All schemas (learner_profile, curriculum_plan, section, textbook)
- BaseProvider + factory + Anthropic + OpenAI implementations
- PipelineNode base class
- TextbookAgent orchestrator
- All 6 pipeline nodes
- HTML renderer with design system
- CLI entry point
- 3 test fixtures + test suite

Success criteria:
`python main.py --profile tests/fixtures/stem_beginner.json` produces a complete, quality-checked HTML textbook with no errors.

Phase 2 — Rich Media

- Image generation via external API (DALL-E 3 or equivalent) for sections needing richer visuals
- Audio narration script generation per section
- Text-to-speech integration (ElevenLabs or equivalent)
- PDF export via WeasyPrint

Phase 3 — Executable Notebooks

- Swap static code blocks for Pyodide-embedded runnable cells
- Output cells appear below code on execution
- State persists between cells within a session
- No server required — runs entirely in browser

Phase 4 — Diagnostic Integration

- Replace the minimal learner profile with the full 8-Steps diagnostic output
- Learner profile populated by the diagnostic system, not manually

- Textbook generation triggered automatically after session completion
- Gap severity and learning signals drive section depth and example density

Phase 5 — Service API

- Wrap the agent in a FastAPI service
- `POST /generate` accepts profile JSON, returns textbook URL
- Async job queue for long-running generation
- Webhook support for completion notification
- Plugs into any interface: chatbot, school portal, mobile app

---

11. Dependencies

```
# requirements.txt
anthropic>=0.30.0
openai>=1.35.0
pydantic>=2.0.0
pydantic-settings>=2.0.0
jinja2>=3.1.0          # HTML templating
prism-python           # code highlighting (or use highlight.js via CDN)
python-dotenv>=1.0.0
pytest>=8.0.0
pytest-asyncio>=0.23.0
```

---

12. Environment Variables

```bash
# .env.example
PROVIDER=claude
ANTHROPIC_API_KEY=sk-ant-...
OPENAI_API_KEY=sk-...
CLAUDE_MODEL=claude-sonnet-4-6
OPENAI_MODEL=gpt-4o
OUTPUT_DIR=outputs/
QUALITY_CHECK_ENABLED=true
MAX_RETRIES=2
TEMPERATURE=0.3
```

13. Notes for the Implementing Agent

Read this before writing any code.

1. **Build schemas first, before any other code.** Everything else is downstream of the schemas being correct. If a schema is wrong, every node built on top of it is wrong.
2. **BaseProvider is the second thing to build.** No node should be written until the provider interface exists. Nodes call providers — if the provider doesn't exist, you will hardcode API calls into nodes and create debt that is painful to remove.
3. **The base pedagogical prompt is sacred.** It must be injected into every content-generating node. Never let a node call an LLM without it. The prompt is what makes this system different from a generic text generator.
4. **Structured output is non-negotiable.** Every LLM call must return a validated Pydantic instance. No string parsing. No regex. If the model cannot conform to the schema, retry. If it still cannot after max_retries, raise a typed error.
5. **The quality checker must re-run only failed nodes.** Do not regenerate the entire textbook when one section fails. The curriculum plan and passing sections are expensive to generate. Only re-run what the quality checker flagged.
6. **The renderer has no LLM calls.** If you find yourself putting an LLM call in the renderer, stop. The renderer is mechanical assembly. All intelligence lives in the pipeline nodes.
7. **Keep the layers separate.** Application layer (`app/`) orchestrates. Service layer (`services/`) implements business logic. Infrastructure layer (`infra/`) handles external systems. No layer reaches past its neighbour.
8. **The design system lives in one CSS file.** All visual decisions are made there. No inline styles in the renderer. No per-textbook visual customisation in Phase 1.
9. **Test against all three fixtures.** A beginner profile, an intermediate profile, and an advanced profile produce very different curricula. The planner must handle all three correctly before Phase 1 is complete.
10. **outputs/ is gitignored.** Generated textbooks are not source code.

End of specification. Phase 1 implementation can begin immediately from this document.